Battle Bots 101

An Introduction Battle Bot Design

August 26, 2026

Timothy S. Leishman
Clinical Assistant Professor
Robotics Engineering Technology
College of Technology
Idaho State University

Contents

Welcome

0.1 What to Expect

BattleBots combine creativity, engineering, and competition into one high-energy learning experience. In this course, students will design, build, and test a small-scale combat robot while learning core principles of mechanical design, electronics, manufacturing, and wireless control systems. Using industry-relevant tools and processes, students will move from concept to a fully functional robot capable of competing in a BattleBot-style event.

Students will gain hands-on experience with 3D CAD using Autodesk Fusion, additive manufacturing (3D printing), electric motors and actuators, and wireless control systems using the ESP32 and Bluetooth Low Energy (BLE) paired with a gaming-style controller. Emphasis is placed on iterative design, problem-solving, teamwork, and safe engineering practices.

By the end of the course, students will not only understand how BattleBots work—but how to design, build, troubleshoot, and improve complex electromechanical systems under real-world constraints such as weight limits, power budgets, durability, and competition rules.

0.2 About the Author

Tim Leishman is an Assistant Professor in the Robotics and Communications Systems Technology program at Idaho State University's College of Technology. He brings over 30 years of professional experience in electronics and systems integration, along with more than a decade of teaching experience at the postsecondary level. His professional background spans industry, military service, and academia, including experience as a Certified Biomedical Equipment Technician (CBET), a field service technician with Tektronix, and service in the United States Air Force supporting advanced electronic systems.

His teaching and technical interests include robotics, embedded systems, motor control, power electronics, CAD and rapid prototyping, and applied wireless communications. Tim is passionate about hands-on, project-based learning and enjoys developing curriculum that bridges theory and real-world engineering practice. He is particularly interested in using competitive robotics, such as BattleBots, to engage students and build practical problem-solving skills.

Chapter 1

BattleBot Fundamentals and Safety

Combat robotics is inherently exciting but also inherently dangerous, which makes safety and rule compliance core fundamentals of any BattleBot-style competition. Successful participation begins with an organized check-in and safety inspection process that ensures every robot meets basic criteria such as acceptable weight class, secure battery containment, clearly identifiable power switch, and an effective weapon lock to prevent accidental activation outside the arena. At load-in, procedures are designed so that robots are only armed once inside a sealed combat zone, mitigating risk to participants and spectators. SPARC (Standardized Procedures for the Advancement of Robot Combat) further codifies safety expectations by requiring that all event robots undergo formal inspection, can be fully deactivated quickly, and are activated only in designated areas with consent from safety officials. Participants are obligated to disclose all operating principles and hazards during inspection, and adherence to these safety practices is critical — noncompliance may result in disqualification or removal from competition. Emphasizing safety from design through competition not only protects individuals and equipment, but also fosters an environment where innovation and strategic engineering can thrive.

1.1 Reference Material:

- Duke University Competition Rules and Procedures
- Standardized Procedures for the Advancement of Robot Combat (SPARC)

Chapter 2

CAD - Intro to Fusion

2.1 Installing Autodesk Fusion (Free Student Version)

Autodesk Fusion is available at no cost to students and educators through Autodesk's Education program. Follow the steps below to install Fusion on your personal computer.

System Requirements

Before beginning the installation, ensure the following requirements are met:

- Windows or macOS computer (Chromebooks and tablets are not supported)
- Reliable internet connection
- At least 8 GB of RAM recommended
- Administrator privileges to install software

Step 1: Create an Autodesk Education Account

1. Open a web browser and navigate to `https://www.autodesk.com/education`
2. Select **Get Started** or **Sign In**
3. Create an Autodesk account using your school-issued email address
4. Verify your student status by entering your school name and expected graduation date

Note: Personal email accounts may not qualify for free educational access.

Step 2: Access Fusion for Education

1. After signing in, navigate to the **Education Products** section
2. Locate **Autodesk Fusion**
3. Select **Fusion – Education**
4. Click **Get Product**

Step 3: Download and Install Fusion

1. Select your operating system (Windows or macOS)
2. Download the Fusion installer
3. Run the installer and follow the on-screen prompts
4. Accept the license agreement and complete the installation

Installation time may vary depending on system performance and internet speed.

Step 4: Sign In and Activate License

1. Launch Autodesk Fusion after installation
2. Sign in using your Autodesk Education account
3. Fusion will automatically activate the educational license

Step 5: Verify Successful Installation

Confirm that the installation was successful by checking the following:

- Fusion launches without license or trial warnings
- A new design file can be created
- Design workspace tools (Sketch, Solid, Modify) are visible

Troubleshooting Tips

- Restart your computer if Fusion does not launch after installation
- Ensure graphics drivers are up to date
- Sign out and back into Fusion if license issues occur
- Educational licenses must be renewed annually

2.2 General / Navigation

Key	Action
S	Toolbox / Shortcut menu
Ctrl + S	Save
Ctrl + Z / Y	Undo / Redo
F6 / F7 / F8	Orbit / Pan / Zoom
Middle Mouse	Zoom / Orbit
Esc	Cancel command

Sketch Mode

Key	Action
L	Line
R	Rectangle
C	Circle
A	Arc
D	Dimension
X	Construction toggle
T	Trim
P	Project
O	Offset

Solid Modeling

Key	Action
E	Extrude
Q	Press Pull
F	Fillet
H	Hole
M	Move / Copy
C	Combine
R	Revolve

Assembly / Components

Key	Action
J	Joint
Ctrl + G	New Component
M	Move Component
Ground	Fix component in place

Inspect / Analyze

Key	Action
I	Measure / Inspect
Ctrl + I	Interference Check
Section Analysis	Cutaway view

2.3 Hello Lego!

Welcome to your first CAD adventure! In this lab, you'll start small but mighty by designing a classic 2×4 LEGO-style brick using Autodesk Fusion. While the part itself is simple and familiar, it's the perfect playground for learning core CAD skills you'll use all semester. You'll create sketches using rectangles and circles, apply dimensions to control size and accuracy, and turn 2D ideas into 3D parts using extrusion. Along the way, you'll explore powerful tools like patterns to repeat features, shell to hollow the brick, and fillets to soften edges for better prints. By the end of the lab, you'll export your design as a .3MF file and send it to a 3D printer—taking your first step from digital design to a physical object you can actually hold. Small brick, big skills unlocked.

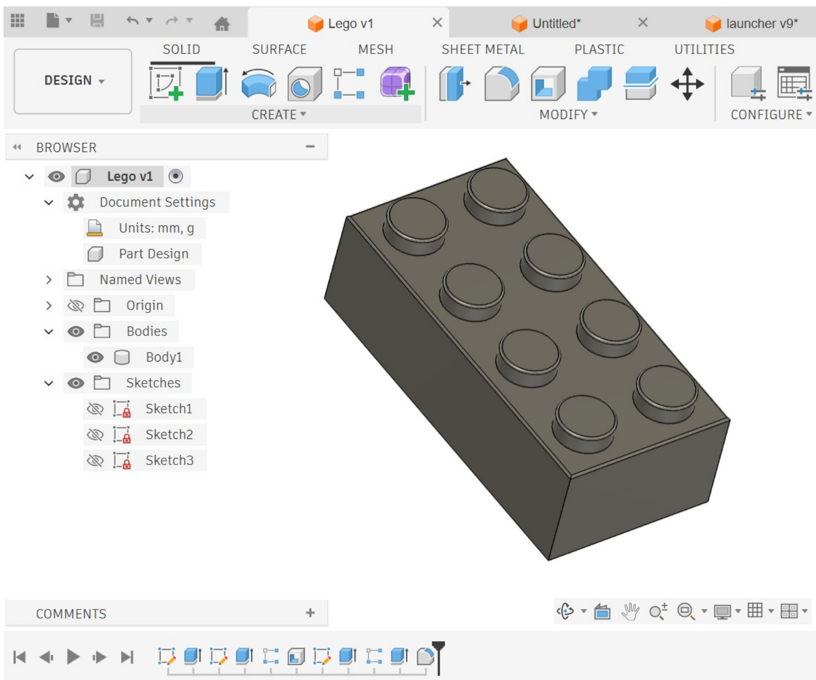


Figure 2.1: Fusion 2x4 Lego

Instructional Content:

- Download 2D Drawing
- Instructional Video

2.4 Thread Carefully

In this project, we scale things up—literally. Welcome to “Thread Carefully,” where you’ll design an oversized bolt and matching nut while learning how precision parts are truly engineered to fit together. This lab introduces more advanced sketching tools like polygons, offsets, mirrors, and construction geometry using midplanes and offset planes to build symmetry and accuracy into your design. You’ll apply Fusion’s thread feature to create realistic mechanical threads, then take it a step further by using the *Combine → Cut → Keep Tools* workflow to physically generate matching internal threads inside the nut using the bolt body itself. By the end of this lesson, you’ll understand how threaded components are modeled, how parametric relationships control fit, and how multiple bodies interact in a professional CAD workflow. Bigger part, tighter tolerances, stronger skills.

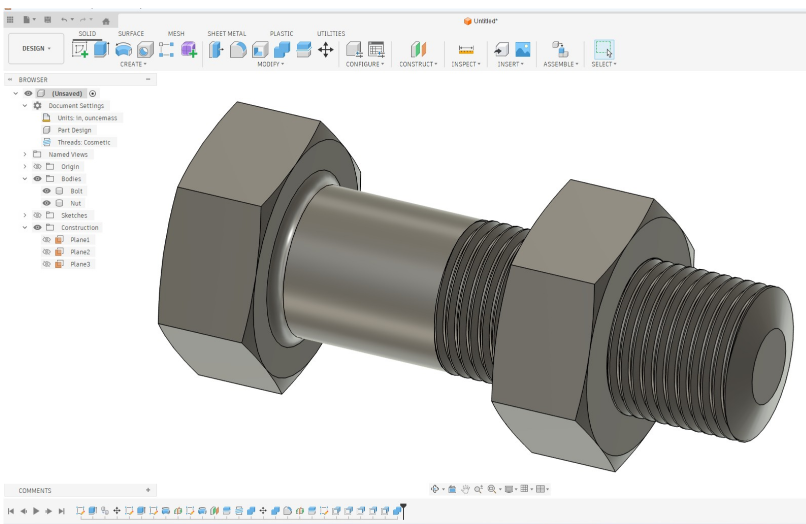


Figure 2.2: Fusion Bolt

Instructional Content:

- Download 2D Drawing
- Instructional Video

2.5 Roll the Model

Welcome to Roll the Model, where precision meets probability. In this lab, you'll design two versions of a die — a classic cube and a beveled backgammon-style version—while reinforcing core sketching and feature-based modeling skills. You'll build clean base geometry, create a sphere to subtract material, and use the Move tool to position dimples accurately. The Pattern feature will allow you to generate a structured 3×3 grid on each face, and from there you'll refine each side by removing unnecessary features to create the correct numbered layout. To complete the project, you'll adjust the Appearance to give your die a realistic finish and then scale the model to produce an oversized 3D-printable version. This exercise strengthens your understanding of symmetry, feature control, pattern management, and visual presentation—proving that even a simple object can be a powerful tool for mastering precision design.

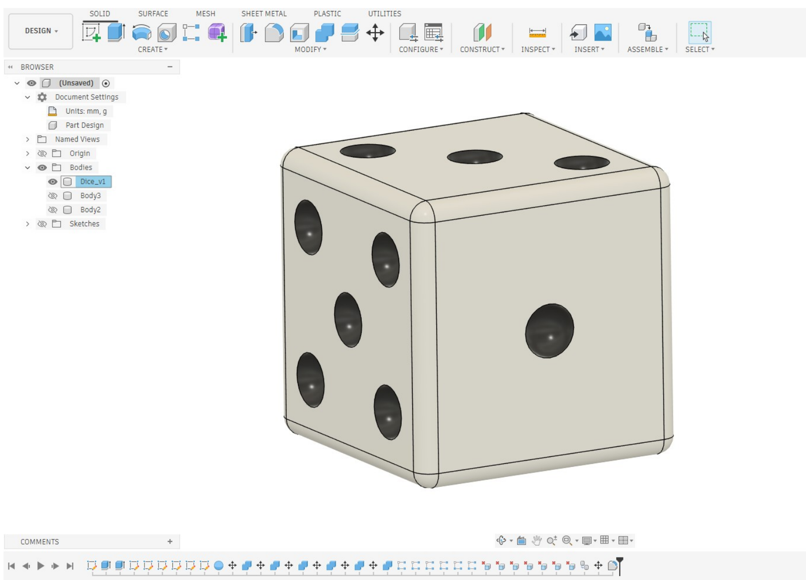


Figure 2.3: Fusion Dice

- Download 2D Drawing
- Instructional Video part 1
- Instructional Video part 2

2.6 Modeling Motion

In this project, we move beyond simple geometric shapes and begin modeling a real-world electromechanical component: the Micro Servo 9G (SG90). Using a combination of manufacturer specifications, online reference dimensions, and physical measurements taken with calipers, you will create an accurate parametric model of the servo housing, output shaft, and horn. This exercise reinforces precision sketching, dimension control, and design intent while introducing the importance of modeling components for real assembly use. After completing the geometry, you will convert the bodies into components and apply a revolute joint to simulate realistic servo rotation. The project concludes with driving and animating the joint to visualize how the servo horn operates—bridging the gap between static modeling and functional mechanical design.

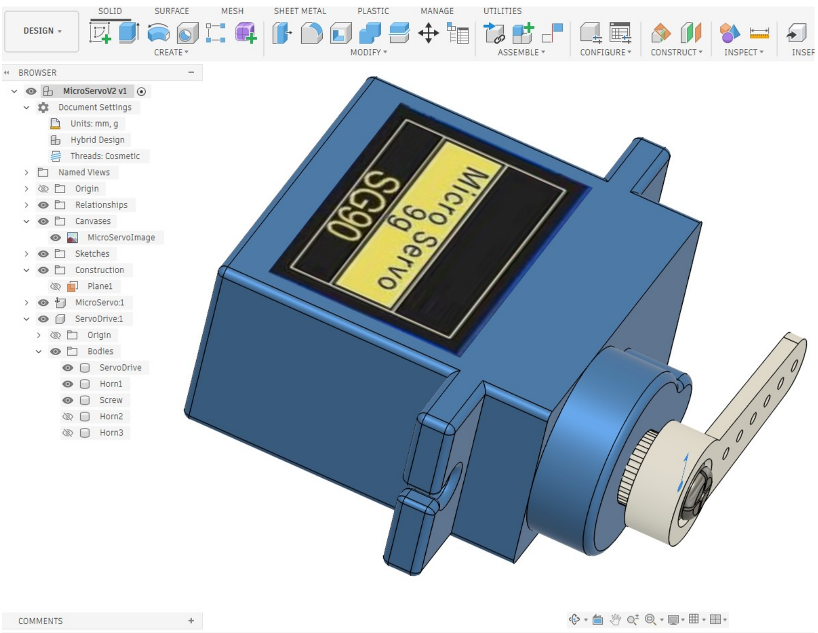


Figure 2.4: Fusion MicroServo

- Download 2D MicroServo Drawing
- Download 2D Horn Drawing
- Instructional Video part 1
- Instructional Video part 2

Chapter 3

Introduction to Motors

3.1 What is an Actuator?

Actuators are the devices that convert electrical energy into controlled physical motion, forming the critical link between electronic control systems and the mechanical world. In modern engineering systems—ranging from robotics and automation to aerospace and consumer electronics—actuators enable movement, positioning, force application, and precise mechanical response. Whether operating as electric motors, linear actuators, solenoids, or servomechanisms, they respond to input signals from controllers and translate those signals into rotational or linear motion. Understanding actuators is essential for designing systems that not only process information, but also interact with their environment in a deliberate, measurable, and repeatable way.

- What is an Actuator?

3.2 Motor Identification

3.2.1 Brushed DC Motor

One of the easiest ways to identify a DC brushed motor is by examining its external features. Most brushed DC motors have two electrical terminals or wires that supply power directly to the motor. Inside the motor are carbon brushes and a commutator, which switch current through the motor windings as the rotor spins. The motor typically has a cylindrical metal housing, and the shaft protrudes from one end for attaching gears, wheels, or pulleys.

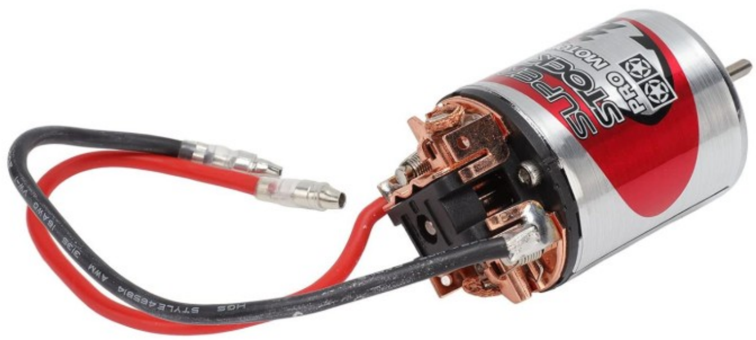


Figure 3.1: Brushed DC Motor Example 1

Figure 3.2: Brushed DC Motor Example 2

3.2.2 Brushless DC Motor

A DC brushless motor (BLDC) can typically be identified by its wiring, construction, and control requirements. Unlike a brushed DC motor, which usually has only two wires and can run directly from a DC power source, a brushless motor normally has three power wires that connect to an Electronic Speed Controller (ESC). These three wires correspond to the motor's three-phase windings, which are energized in sequence by the ESC to produce rotation. Brushless motors also lack internal brushes and a commutator; instead, they contain permanent magnets on the rotor and electromagnetic coils in the stator, with commutation handled electronically by the controller. In many hobby and robotics applications—such as drones and small robotic systems—the motor may also have a ventilated housing or a rotating outer casing (an outrunner design). Because of their higher efficiency, longer lifespan, and precise electronic control, brushless motors are widely used in modern robotics, drones, electric vehicles, and other high-performance electromechanical systems.

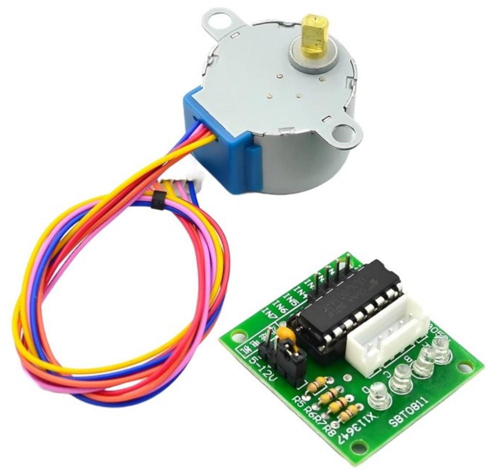
Figure 3.3: Brushless DC Motor Example 1

Figure 3.4: Brushless DC Motor Example 2

3.2.3 Stepper Motor

A stepper motor can be identified by its wiring, construction, and the way it is controlled. Unlike standard DC motors that spin continuously when power is applied, a stepper motor moves in precise, discrete steps, allowing accurate control of position and rotation. One of the most noticeable features is the number of wires, which is typically four or five depending on whether the motor is bipolar or unipolar. These wires connect to multiple internal coils that are energized in sequence by a stepper motor driver, rather than directly from a simple DC power supply. Stepper motors often have a square or rectangular metal body, commonly seen in NEMA-style packages used in 3D printers, CNC machines, and robotics. Inside the motor are multiple electromagnetic coils arranged around a rotor that contains permanent magnets or a toothed iron core. By energizing the coils in a specific order, the driver causes the rotor to move one step at a time, making stepper motors ideal for applications requiring precise positioning, repeatable motion, and open-loop control.

(a) BiPolar Stepper Motor Example



(b) UniPolar Stepper Motor Example

Figure 3.5: Stepper Motors

3.2.4 Servo

A servo is a self-contained actuator designed for precise control of position, speed, or angular displacement. Unlike a simple motor that spins continuously when powered, a servo integrates a motor, gear train, feedback mechanism (typically a potentiometer or encoder), and a control circuit into one compact unit. By using feedback to compare a commanded position with its actual position, the servo automatically corrects error and holds its target angle with accuracy and stability. Servos are widely used in robotics, automation, RC systems, and mechatronic projects because they provide reliable, repeatable motion control using simple control signals—often just a pulse-width modulation (PWM) input from a microcontroller.

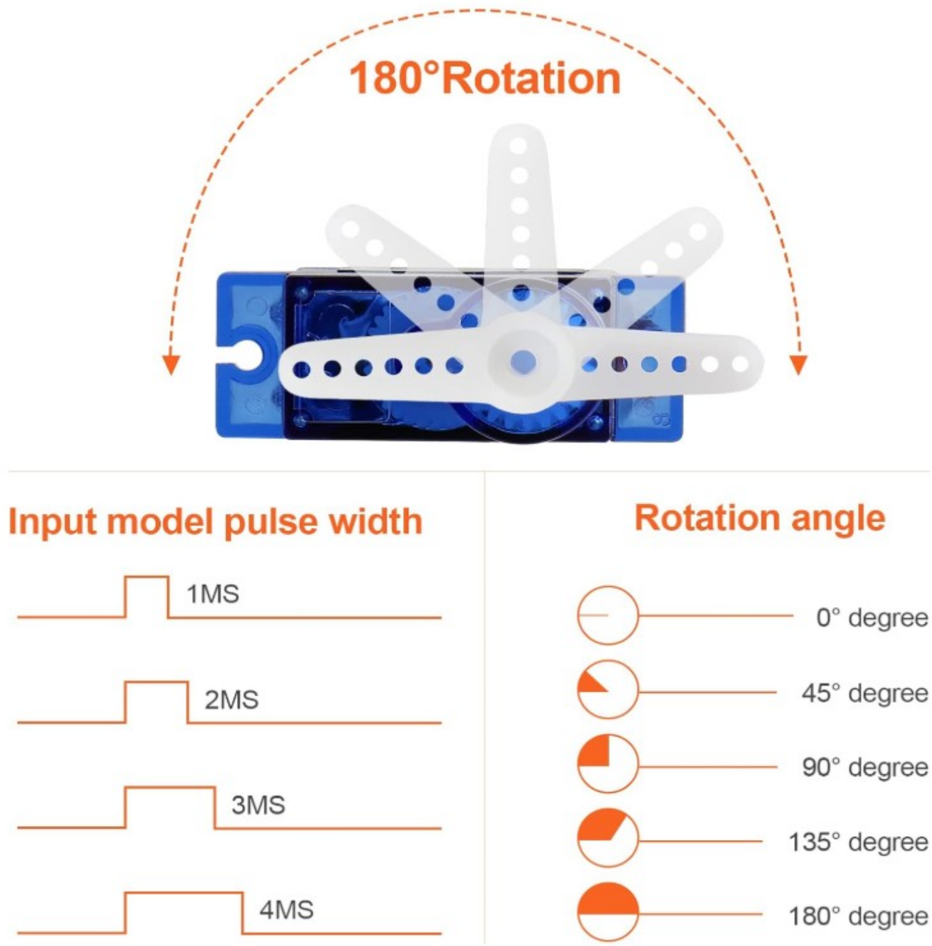


Figure 3.6: Servo Image with PWM Control Signal

Chapter 4

Motor Control Test Circuits

4.1 Brushed DC Motors

4.1.1 Evolution of the H-Bridge

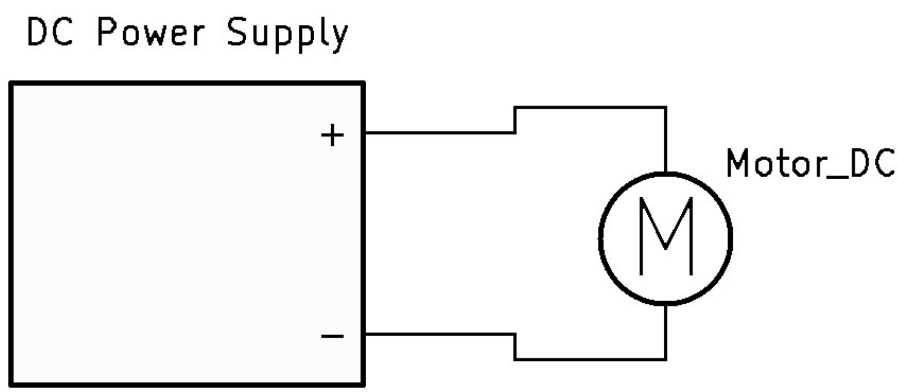


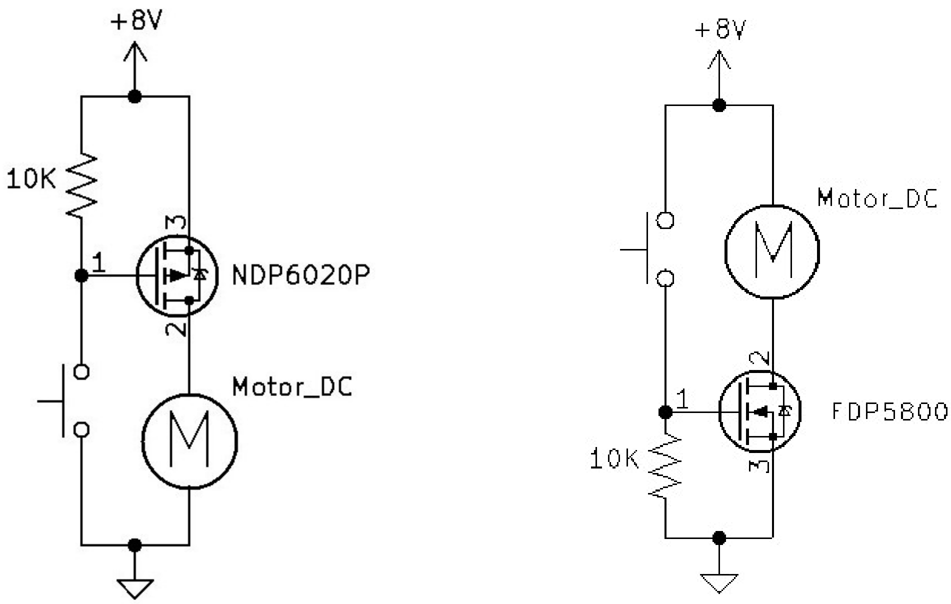
Figure 4.1: DC Motor Test Circuit 1

Test 1

(See Figure 4.1: DC Motor Test Circuit 1)

- Set the DC Power Supply to the motor's rated voltage
- Observe and note the current and RPM at the motor's rated voltage
- Observe the rotation (clockwise or counter-clockwise?). Swap the DC voltage polarity to the motor wires and observe the rotation.

Positive and Negative voltage control



(a) P-Channel MOSFET Control **(b)** N-Channel MOSFET Control

Figure 4.2: H-Bridge Test Circuits Part 1

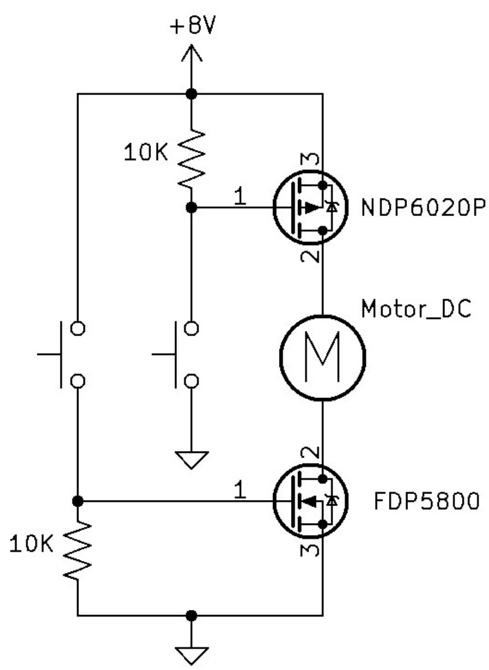


Figure 4.3: P-Channel and N-Channel with two momentary switches

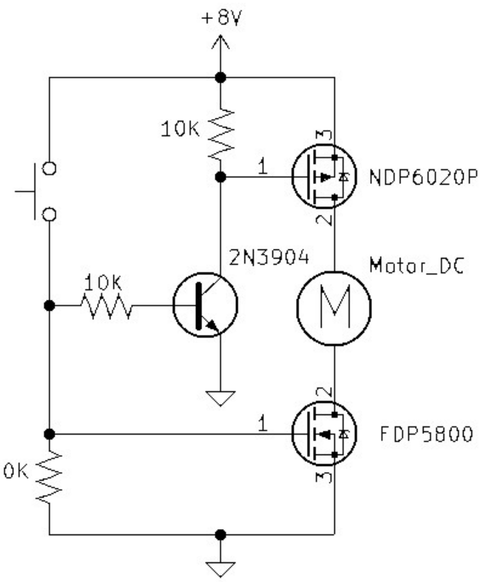


Figure 4.4: P-Channel and N-Channel with one momentary switch and a 2N3904

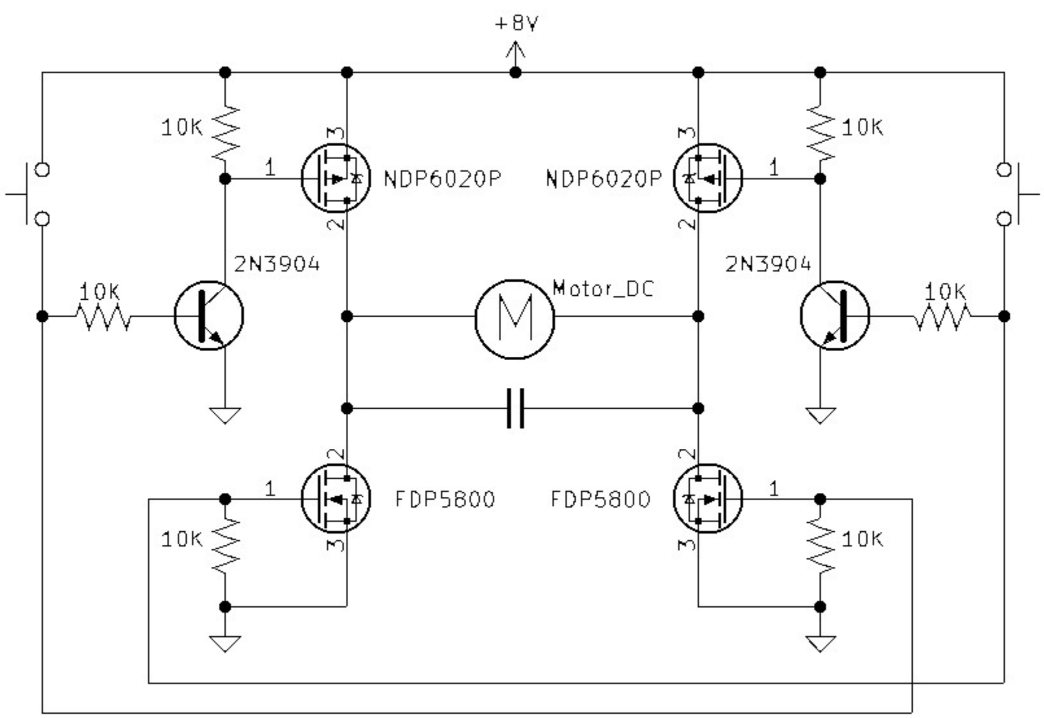


Figure 4.5: H-Bridge Test Circuit 1

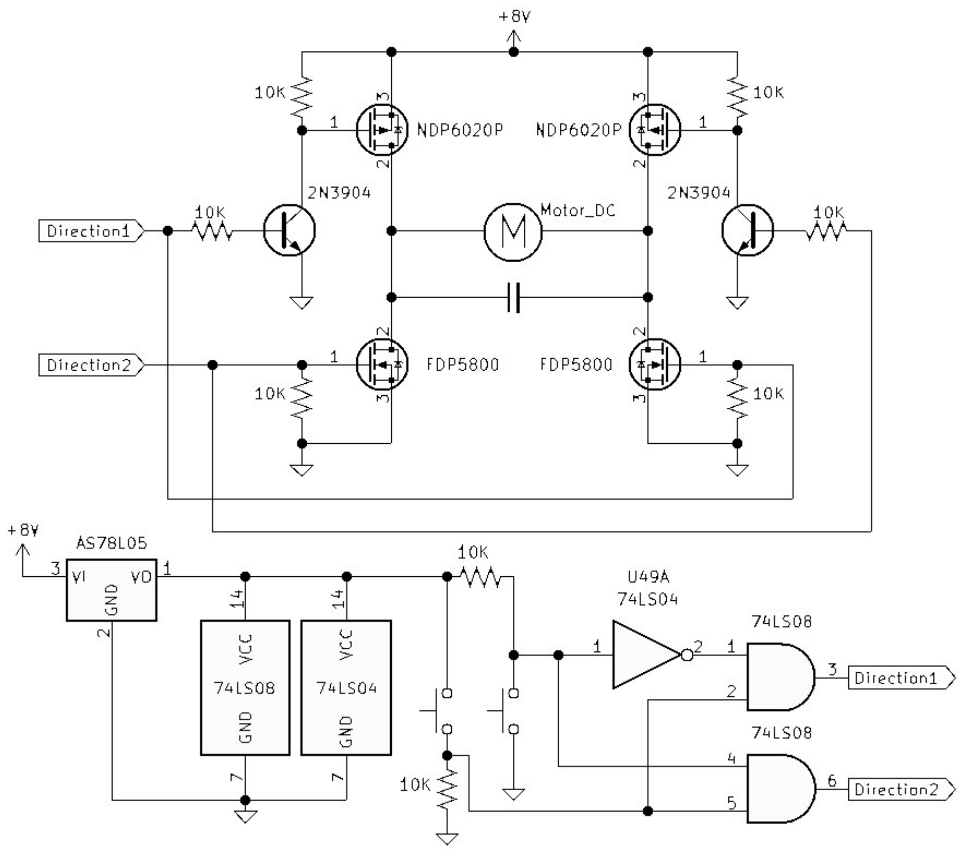


Figure 4.6: H-Bridge Test Circuit 2

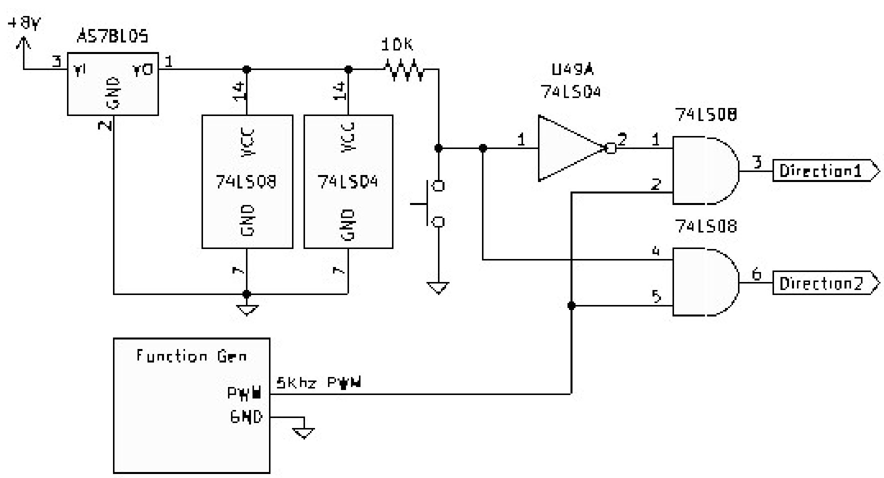


Figure 4.7: H-Bridge Test Circuit 2 modified with PWM Speed Control

4.1.2 Alternative 555 Timer PWM Test Circuit

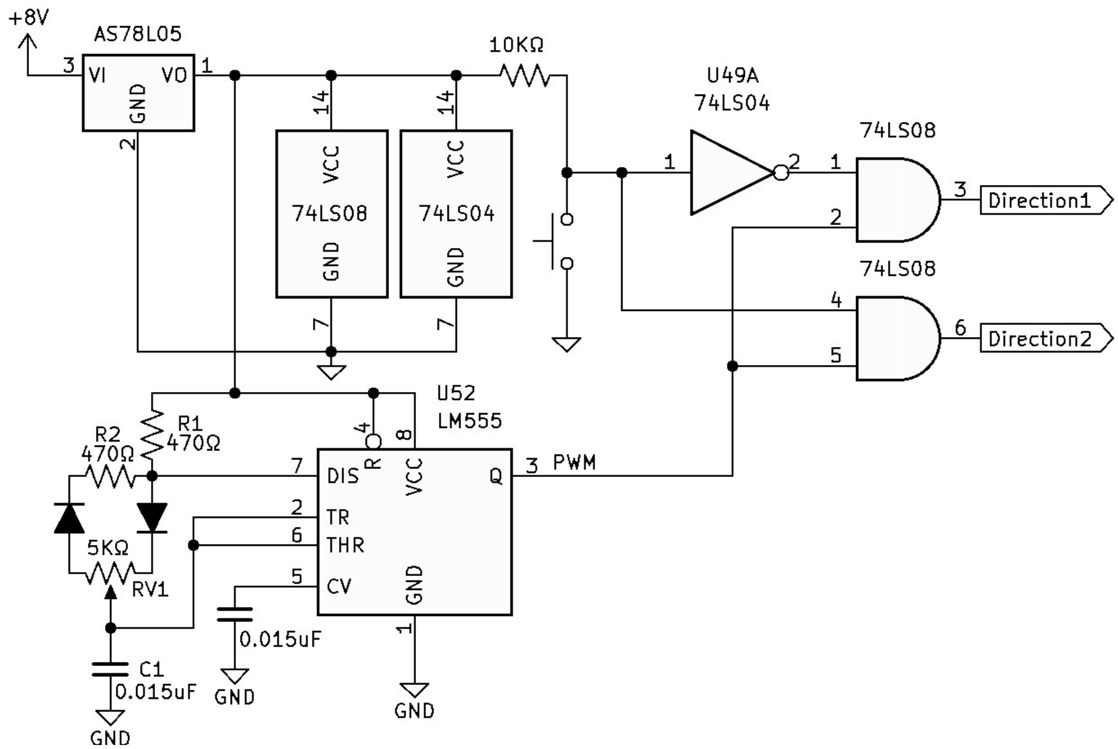


Figure 4.8: H-Bridge Test Circuit 2 modified with 555 timer PWM Speed Control

1. Calculate Circuit Components.
   - $Time_{On} \approx 0.693(R1 + RV1a)(C1)$
   - $Time_{Off} \approx 0.693(R2 + RV1b)(C1)$
   - Calculate circuit C1, R1, and R2.
     - Calculate for 5Khz at 50% DC with a 5KΩ variable potentiometer, $Time_{On} = 200\mu S$ and $Time_{Off} = 200\mu S$.
     - Make $R1$ & $R2 \approx \frac{RV1}{10}$
       - $R1$ & $R2 \approx \frac{5K}{10}$
       - $R1$ & $R2 \approx 500\Omega$
     - Imagine the variable potentiometer is centered, making both sides 2.5KΩ.
     - Solve R1 using Simultaneous Linear Equations:
       - First Equation: $200\mu S = 0.693 \times (500 + 2.5K\Omega) \times C1$
       - (Solve for C1) $C1 = \frac{200\mu S}{0.693\times(500+2.5K\Omega)}$
       - $C1 = .0962\mu F$
       - *NOTE: During testing it was found that the PWM frequency was causing an audible frequency reaction with the motor at lower speeds. This was corrected by adjusting the timing capacitor C1 down, which raised the PWM frequency beyond the audible range.

4.1.3 Alternative H-Bridge L293DNE Test Circuit

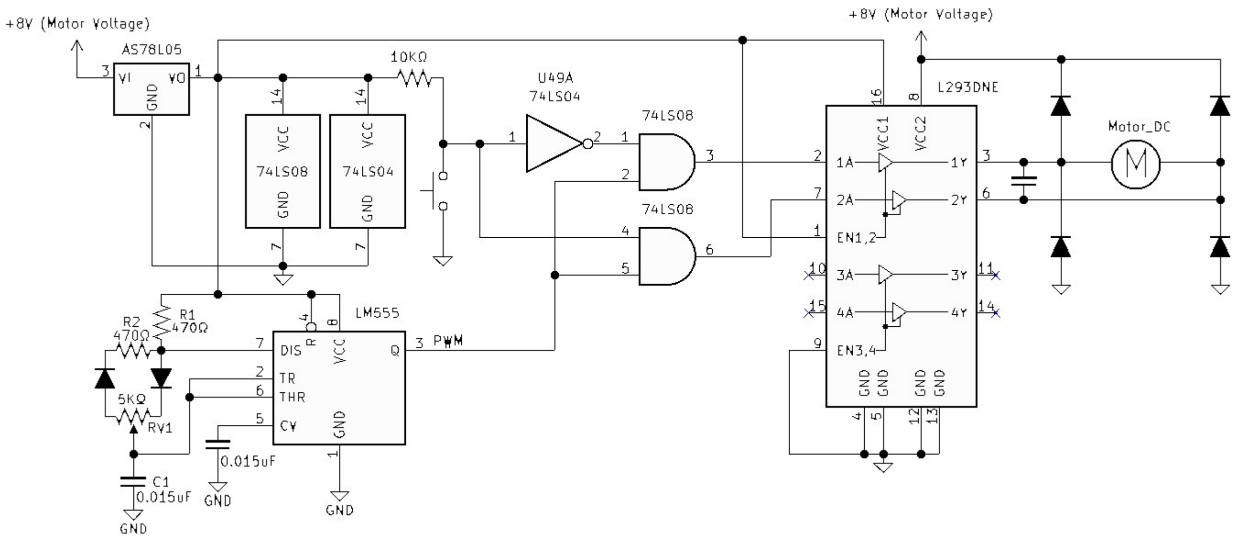


Figure 4.9: H-Bridge Test Circuit 3 modified with L293DNE

4.2 Stepper Motor Control

Small Reduction Stepper Motor Reference Material

- 918 Data Sheet

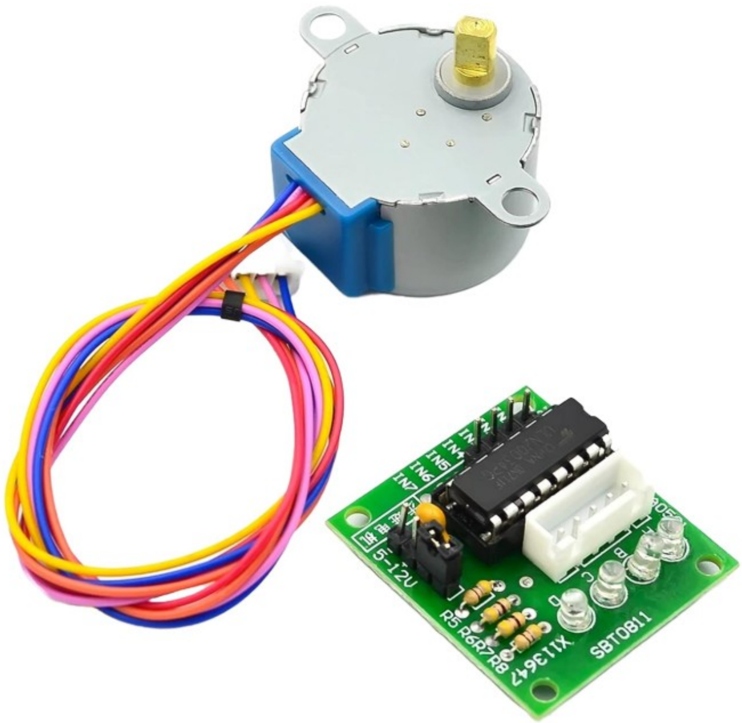


Figure 4.10: UniPolar Stepper Motor

4.2.1 28BYJ-48 (ULN2003) & Ring Counter

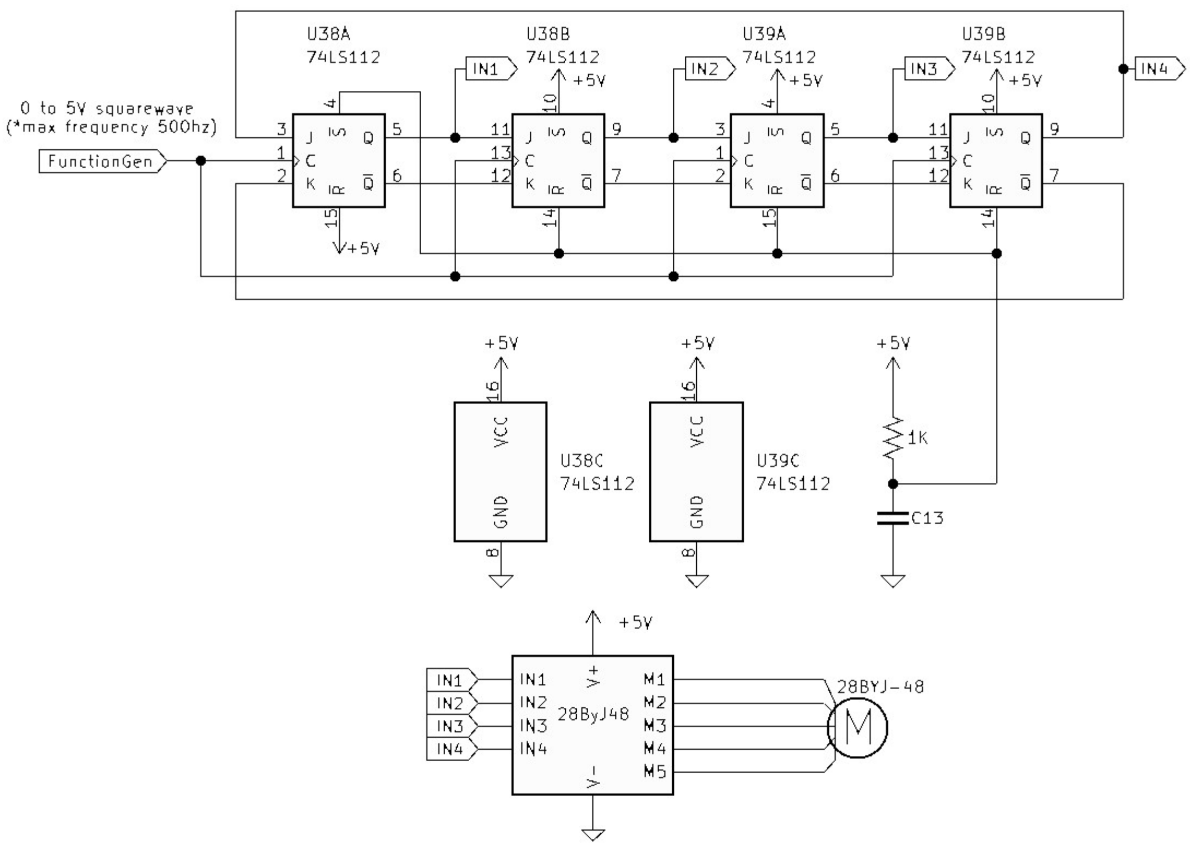


Figure 4.11: 28BYJ-48 /ULN2003 Ring Counter Test Circuit Schematic

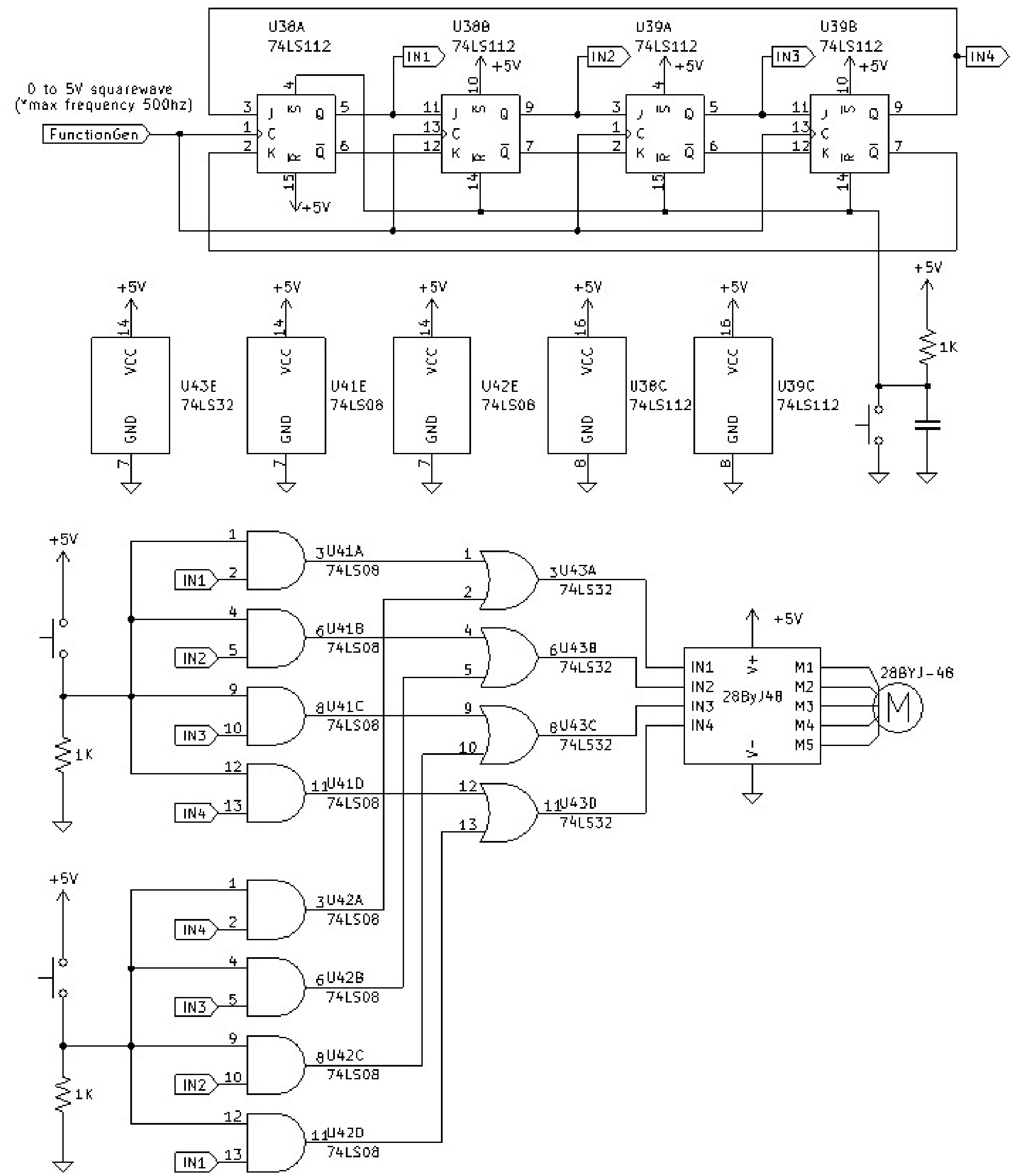


Figure 4.12: 28BYJ-48 /ULN2003 Ring Counter Test Circuit 2 Schematic

4.2.2 28BYJ-48 (ULN2003) & 74LS194 Universal Shift Register

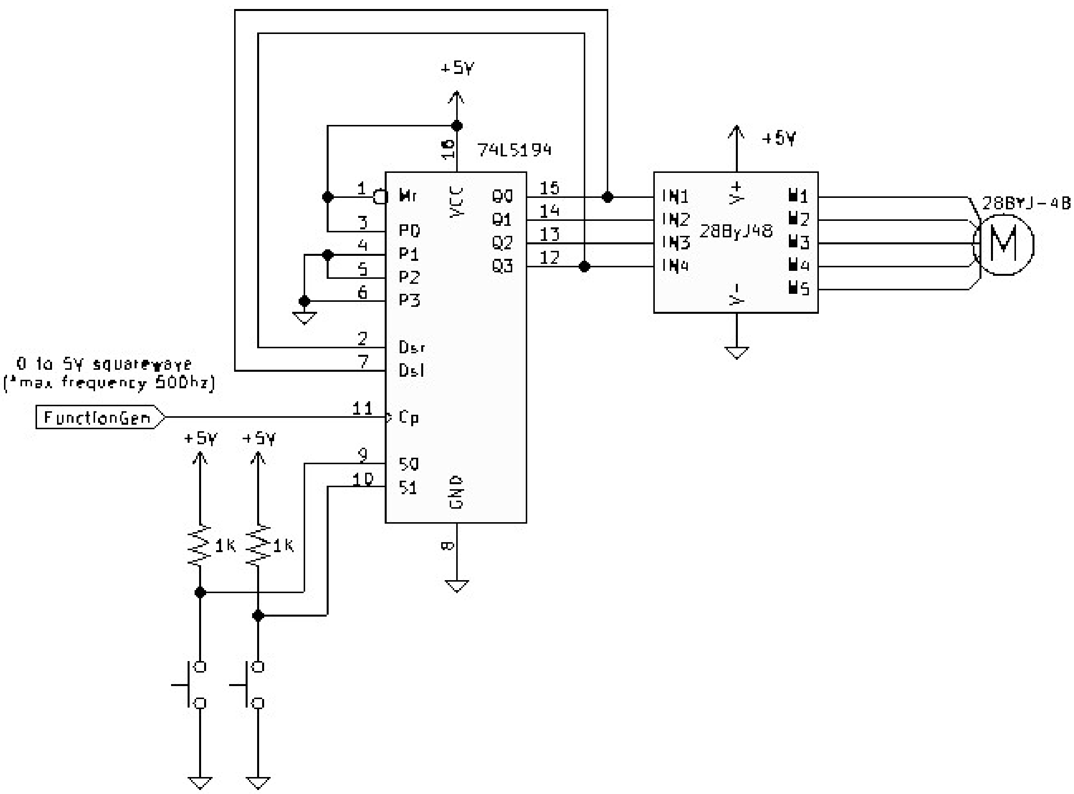
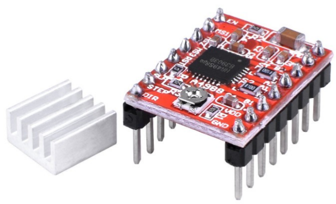
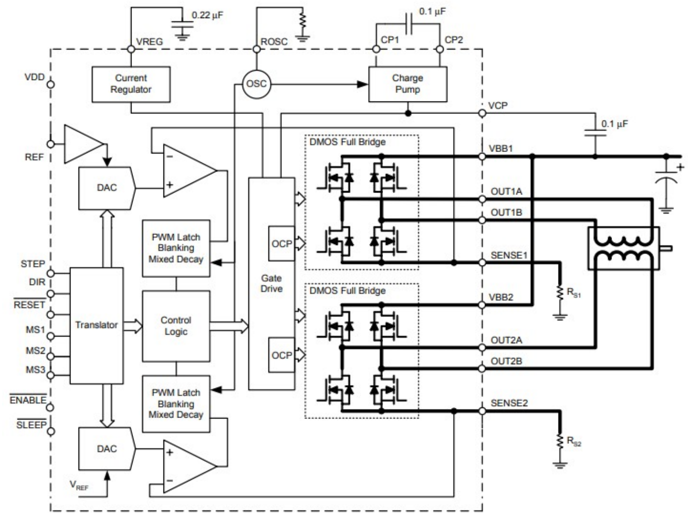


Figure 4.13: 28BYJ-48 /ULN2003 Universal Shift Register Test Circuit Schematic

4.2.3 A4988 (Bipolar Driver)



(a) A4988 Module



A4988 Functional Diagram

Figure 4.14: A4988 Stepper Motor Driver

A4988 Reference Material

- A4988 Overview
- A4988 Data Sheet
- A4988 Current Limiting Calibration Video

Table 1: Microstepping Resolution Truth Table

MS1	MS2	MS3	Microstep Resolution	Excitation Mode
L	L	L	Full Step	2 Phase
H	L	L	Half Step	1-2 Phase
L	H	L	Quarter Step	W1-2 Phase
H	H	L	Eighth Step	2W1-2 Phase
H	H	H	Sixteenth Step	4W1-2 Phase

Figure 4.15: A4988 Microstepping Resolution Table

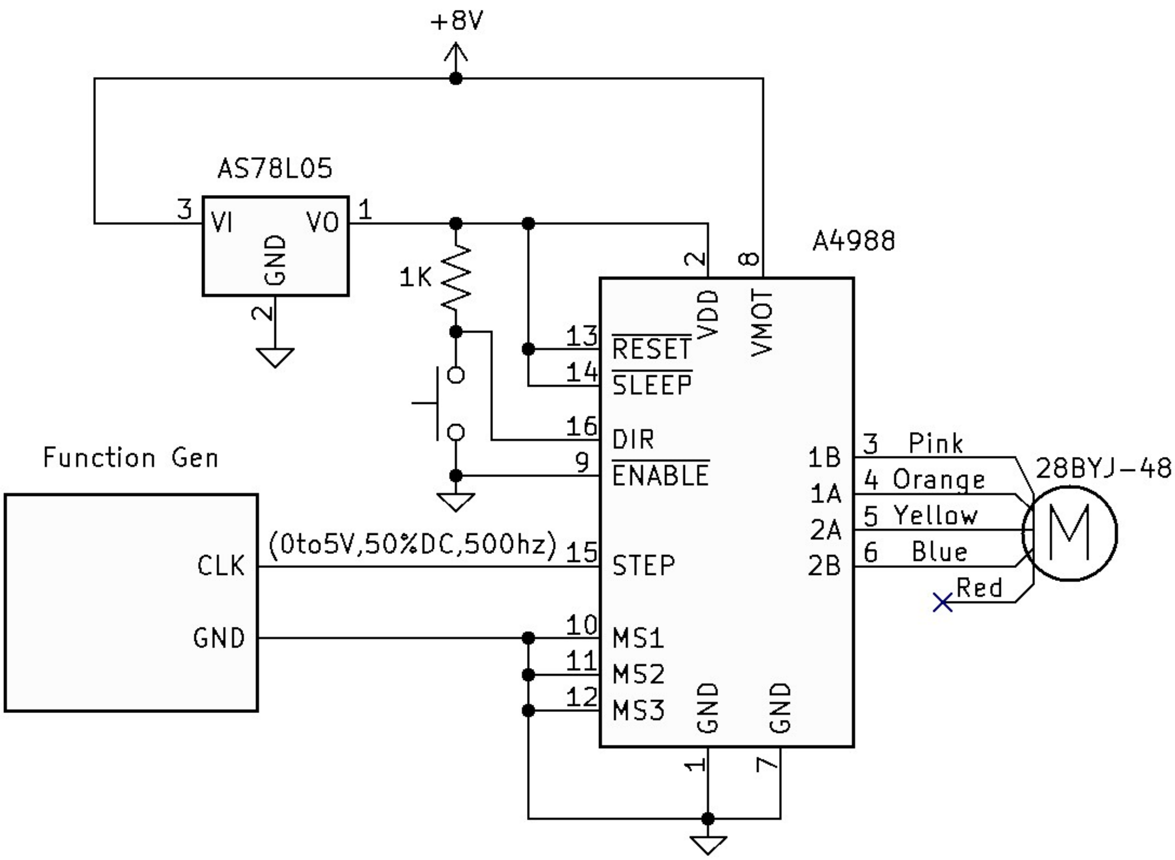


Figure 4.16: A4988 Test Circuit Schematic

4.2.4 TMC2208

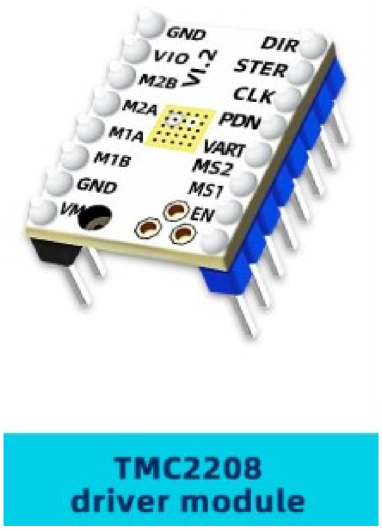


Figure 4.17: TMC2208 Driver Module

TMC2208 Reference Material

- TMC2208 Data Sheet

MS2(-)	MS1(-)	Steps(-)	Interpolation(-)	Mode(-)
GND	VIO	1/2	1/256	stealthChop2
VIO	GND	1/4	1/256	stealthChop2
GND	GND	1/8	1/256	stealthChop2
VIO	VIO	1/16	1/256	stealthChop2

Figure 4.18: TMC2208 Microstepping

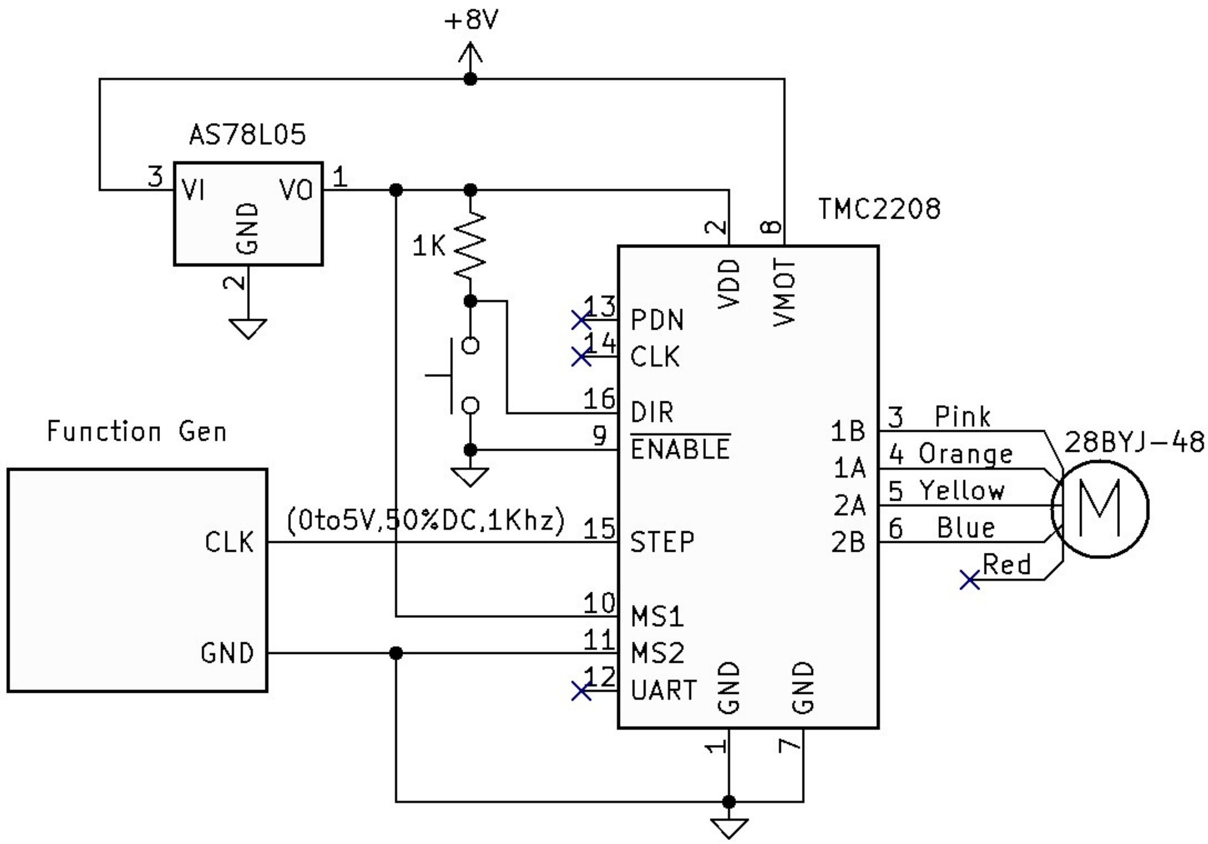


Figure 4.19: TMC2208 Test Circuit Schematic

4.3 Servo

4.3.1 Servo Parallax Documentation

PARALLAX

Web Store: www.parallax.com
Tutorials: learn.parallax.com
Sales: sales@parallax.com
Tech Support: support@parallax.com

Sales: (888) 512-1024
Educator Hotline: (916) 701-8625
Office: (916) 624-8333
Fax: (916) 624-8003

Parallax Continuous Rotation Servo (#900-00008)

The Parallax Continuous Rotation Servo provides bi-direcitonal continuous rotation with a linear response to pulse width. Two are used to power the drive wheels in the following Parallax robotics kits:

- BASIC Stamp Boe-Bot (#28132, #28832, #81031)
- SumoBot, Original (#27400) and WX (#32134)
- Shield-Bot with Arduino (#32335 & 81033)
- cyber:bot with micro:bit (#32700)

This document supports the Digital version of the Parallax Continuous Rotation Servo, released in summer 2022. It is designed to be a drop-in replacement of the previous analog version for the purpose of Parallax robotics; applications like adding this device to a digital bus are not supported. For the analog Parallax Continuous Rotation Servo previously sold, see the product guide version 2.2 available from the 900-00008 product page at www.parallax.com.

For best results, use two of the same type (analog or digital) when selecting or replacing servos for your robot.

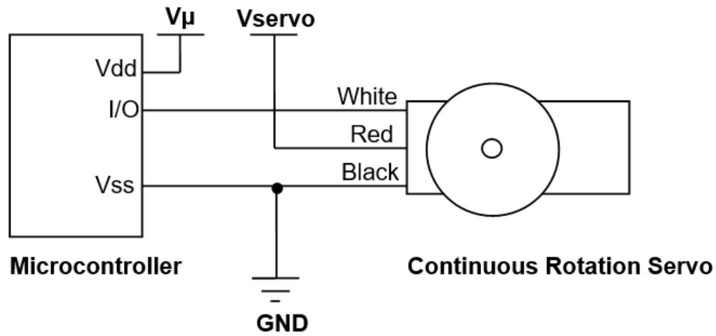
Features

- Bidirectional continuous rotation
- 0 to +/-50 RPM with 5 V supply
- Linear response to pulse-width for easy speed ramping
- Color coded cable (red VDD, white signal, black ground) with 3-pin 0.1" socket
- Accepts four mounting screws
- Easy to interface with any Parallax microcontroller or PWM-capable device

Specifications

- Gear material: POM (polyacetal)
- Spline: 3F 25-tooth
- Weight: 1.50 oz (42.5 g)
- Torque: 38 oz-in @ 6 V
- Power requirements: 4 to 6 VDC; Maximum current draw 140 +/- 50 mA at 6 VDC when operating in no load conditions, 15 mA when in static state
- Communication: pulse-width modulation
- Center pulse width: 1520 µs (before manual centering)
- Zero speed seadband: 12 µs
- Dimensions: approx 2.2 x 0.8 x 1.6 in (5.58x 1.9 x 4.06 cm) excluding servo horn
- Operating temperature range: 14 to 122 °F (-10 to +50 °C)

Electrical Connections



Vµ = microcontroller voltage supply

Vservo = 4 to 6 VDC, regulated or battery

I/O = PWM TTL or CMOS output signal, 3.3 to 5 V; < Vservo + 0.2 V

To use this servo with a Parallax robot, refer to the robot's documentation on learn.parallax.com. To use this servo with other devices, keep the following power precautions and other tips below in mind.

Power Precautions

- Power the servo with regulated 4-6 VDC, with a supply that can provide up to 500 mA as load may spike when the servo starts, stops, or turns. Four alkaline AA batteries or five NiMH rechargeable batteries in series are also acceptable as a supply for these servos.
- Servo current draw can spike while under peak load; be sure your application's regulator is rated to supply adequate current for all servos used in combination.
- Do not use this servo with an unregulated wall-mount supply. Such power supplies may deliver variable voltage far above the stated voltage.
- Do not try to power the servo directly through a microcontroller I/O pin.

Other Tips

- Calibrate (center) the servo with your project's microcontroller before permanently installing it into your application.
- Calibration requires access to the port in the side of the servo's case. For periodic recalibration, you may wish to design your project so the port remains accessible after installation.
- When using two servos for a small robot's drive wheels, use two of the same type (analog or digital).
- When using two servos for a small robot's drive wheels with servo horns pointed outwards, keep in mind that one servo must rotate clockwise while the other rotates counterclockwise at the same rate in order for the robot to drive straight.

Device Information

The Parallax continuous rotation servo relies on pulse width modulation to control the rotation speed and direction of the servo shaft. Before using the servo in a project, it is important to calibrate the servo's center setting in order to define the pulse width value at which the servo holds still (see the section Calibration – "Center" the Servo).

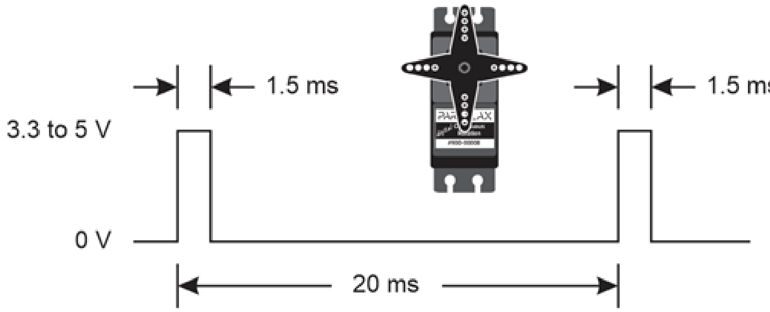
Specifications

Pin	Name	Description	Minimum	Typical	Maximum	Units
1 (White)	Signal	Input; TTL or CMOS	3.3	5.0	Vservo + 0.2	V
2 (Red)	Vservo	Power Supply	4.0	5.0	6.0	V
3 (Black)	Vss	Ground		0		V

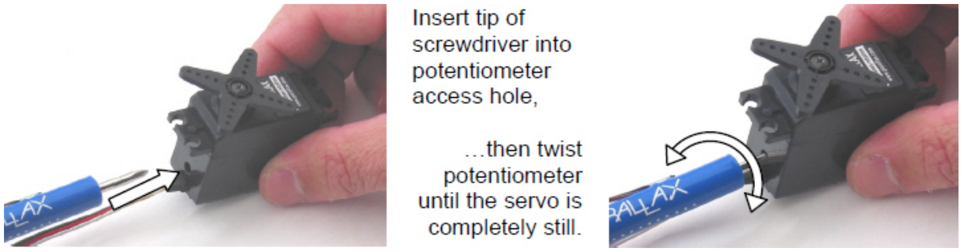
Calibration – "Center" the Servo

In most Parallax small robots, the servos are manually centered. Centering is a procedure that makes the servo stay still in response to a certain control signal.

Step 1: A microcontroller is programmed to send 1.5 ms (1500 μs) control pulses every 20 ms. In response to these pulses, a servo that has not been centered before typically responds by turning its output shaft (and horn) slowly clockwise.



Step 2: A #1 Phillips screwdriver is used to adjust the servo's trim potentiometer to make it stop turning. This establishes the 1.5 ms (1500 μs) as the signal to make the servo stay still.



NOTE: The above procedure does not ensure that the middle of the servo's deadband range is aligned with 1500 µs. For example, instead of staying still in response to pulses in the 1494 to 1506 µs range, it might instead stay still in response pulses in the 1490 µs to 1502 µs range. To more precisely center the servo around 1500 µs pulses, a microcontroller program can be written that repeatedly sends:

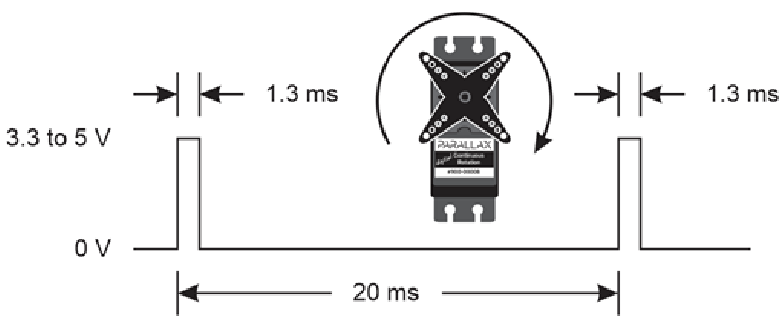
- 1485 µs pulses every 20 ms for 2 seconds
- 1515 µs pulses every 20 ms for 2 seconds.

Adjust the potentiometer with the screwdriver until the slow counterclockwise and clockwise velocities match.

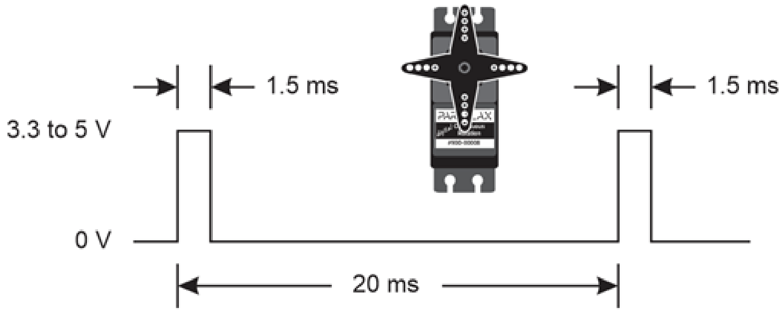
Communication Protocol

The Parallax Continuous Rotation Servo is controlled through pulse width modulation. Rotational speed and direction are determined by the duration of a high pulse, in the 1.3–-1.7 ms range. To rotate smoothly, the servo needs the control pulses to be repeated every 20 ms. Below are example timing diagrams for signals to make the servo turn full speed in both directions as well as stay still (assuming the centering procedure above has been completed):

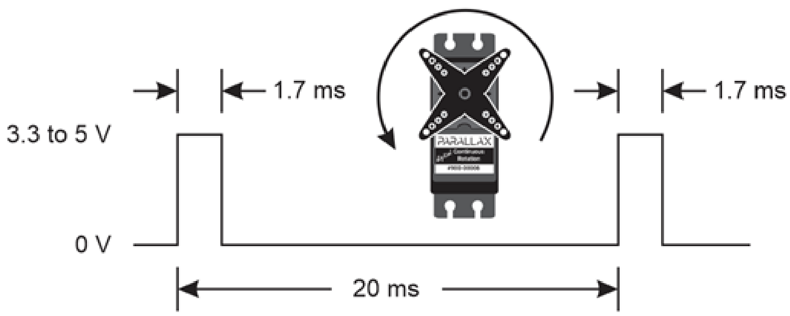
Full speed clockwise:



No rotation:

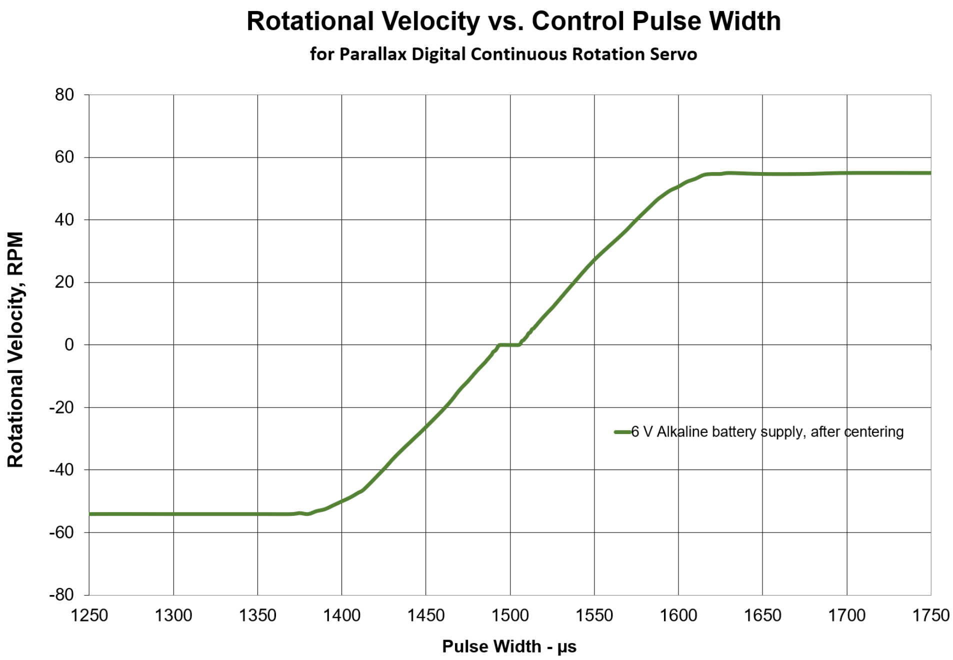


Full speed counterclockwise:



Control Servo Speed with Pulse Widths from its Transfer Function

The transfer function graph below shows the servo's velocity response to pulse width. Positive RPM values represent counterclockwise rotation; negative values represent clockwise rotation. With the servo precisely centered at 1500 µs, counterclockwise rotation speed increases linearly as the control pulse width is swept from 1506 to 1586 µs. Pulse widths at or above 1620 µs result in full speed. Likewise, clockwise rotation speed increases linearly with control pulse widths from 1494 to 1914 µs, and pulses with durations less than 1380 µs result in full speed.

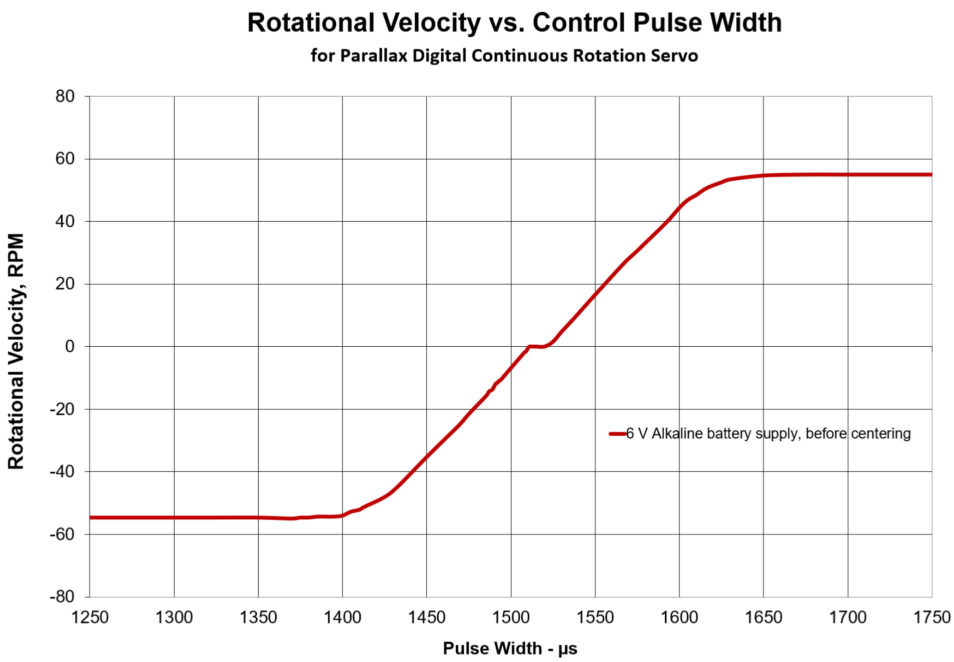


NOTES:

Variations in manufacturing, supply, temperature, and centering will affect your servo's actual values. Particularly, battery pack decay will affect the velocity response. The servos will run faster with new batteries, but the speed will decay as the batteries drain. With these variations in mind, supply voltage and control signal adjustment will be needed to suit your continuous rotation servo application.

Maximum RPM will vary with input voltage; 50 RPM @ 5 V is typical.

Before manual centering, the transfer function for a Parallax Digital Continuous Rotation Servo resembles this plot. The zero speed deadband is typically 12 µs wide, and centered at 1520 µs.



Resources and Downloads

Check for the latest servo documentation and resources on the #900-00008 product page at www.parallax.com. For servo use in a specific Parallax robot see the tutorials at learn.parallax.com.

Revision History

3.0 supports the digital version of the Parallax Continuous Rotation Servo, released in June 2022. See 2.2 for information on the previous analog version of this servo; it is available from the 900-00008 product page at www.parallax.com.

4.3.2 Inexpensive Servos

Figure 4.20: SG90 Micro Servo Standard 180°

Figure 4.21: Continuous Servo FSR90R

4.3.3 Inexpensive Servo Videos

- SG90 Servo Video
- FS90R Continuous Motion Servo Video

4.3.4 Initial Servo Test Steps:

1. Use an Oscilloscope to measure and set a DC power supply output to 5VDC.
2. Setup a Function Generator to produce a 0 to 5v, pulse waveform with a positive pulse of 1.5ms and a 20ms off time ($\approx 50hz$). Use an Oscilloscope to verify the signal.
3. Once the 5VDC and the 0 to 5V pulse waveform has been verified, the servo can be connected as follows.
   - Red to 5VDC
   - Black (sometimes Brown) to Reference
   - White (sometimes yellow) to Function Generator (0 to 5V pulse waveform)
   - Incrementally adjust the pulse width of the waveform from 1.5ms to 2.0ms and back down to 1.0ms and observe the Servo motion.
   - Document the pulse width range for use in the next sections.

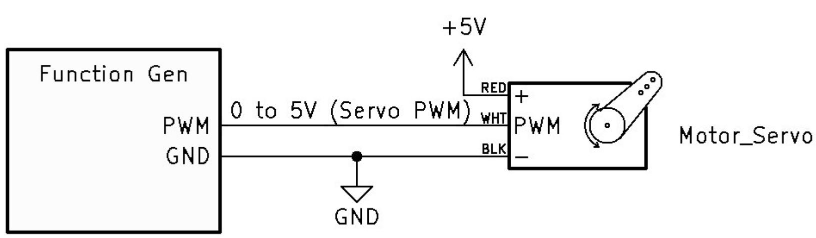


Figure 4.22: Servo Test Circuit Schematic

4.3.5 Alternative 555 Timer Servo Test Circuit

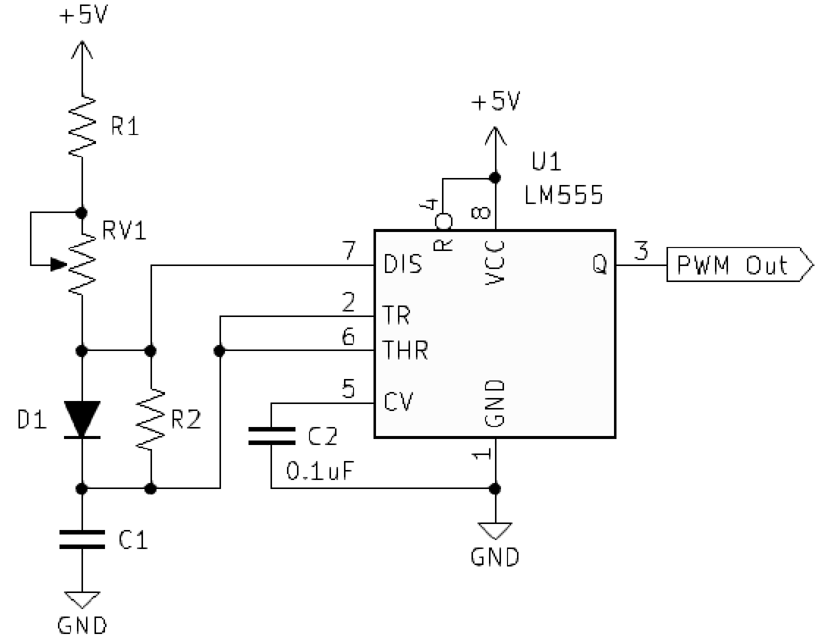


Figure 4.23: 555 Timer Test Circuit

1. Calculate Circuit Components.
   - $Time_{On} \approx 0.693(R1 + RV1)(C1)$
   - $Time_{Off} \approx 0.693(R2)(C1)$
   - Calculate circuit C1, R1, RV1, and R2.
     - Calculate for a variable PW of 0.75mS to 2.25mS.
     - Measure the resistance of the variable resistor you will be using in the circuit, mine is $5K\Omega$.
     - Solve R1 using Simultaneous Linear Equations:
       * First Equation: $0.75mS = 0.693 \times R1 \times C1$
       * (Solve for C1) $C1 = \frac{0.75ms}{0.693 \times R1}$
       * Second Equation: $2.25mS = 0.693 \times (5K\Omega + R1) \times C1$
       * (Solve for C1) $C1 = \frac{2.25mS}{0.693 \times (5K\Omega + R1)}$
       * Solve for R1 using Super Substitution: $\frac{0.75ms}{0.693 \times R1} = \frac{2.25mS}{0.693 \times (5K\Omega + R1)}$
       * $0.75mS(5K\Omega + R1) = 2.25mS(R1)$
       * $3.75 + 0.75mS(R1) = 2.25mS(R1)$
       * $3.75 = 1.5mS(R1)$
       * $R1 = \frac{3.75}{1.5mS}$

    - $R1 = 2.5K\Omega$
  - Solve for C1. (use both equations to verify the R1 calculations)
    - $C1 = \frac{0.75ms}{0.693 \times R1} = \frac{0.75ms}{0.693 \times 2.5K\Omega} = 0.4329\mu F$
    - $C1 = \frac{2.25mS}{0.693 \times (5K\Omega + R1)} = \frac{2.25mS}{0.693 \times (5K\Omega + 2.5K\Omega)} = 0.4329\mu F$
    - $C1 = 0.4329\mu F$
  - Solve for R2:
    - $20mS = 0.693 \times C1 \times R2$
    - $R2 = \frac{20mS}{0.693 \times 0.4329\mu F}$
    - $R2 = 66.667K\Omega$

- Verify the PWM Out waveform using an Oscilloscope. Adjust RV1 and observe the pulse width will vary from 0.75ms to 2.25ms. Lower or raise R1 as needed to achieve the minimum Pulse Width. Lower or raise R2 as needed to achieve the approximate Pulse Space needed.
- Verify that the pulse space is approximately 20ms.
- Verify that the waveform voltage is 0 to 5V.

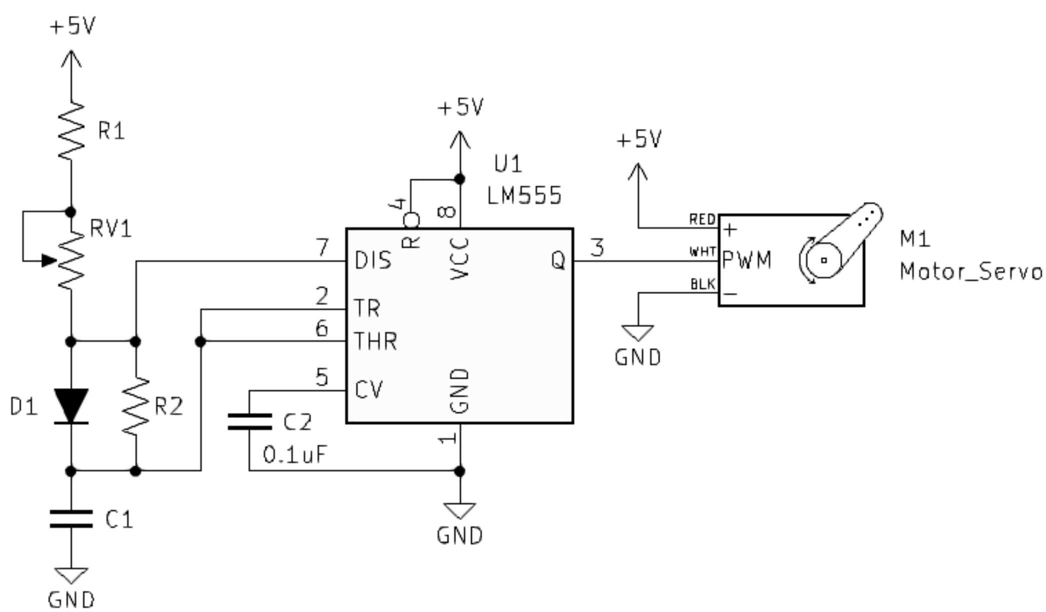


Figure 4.24: 555 Timer Servo Circuit Schematic

2. Test the final circuit with the servo motor and fine-tune the circuit as needed.

4.4 Electronic Speed Control (ESC)

4.4.1 ESC Introduction

An Electronic Speed Controller (ESC) is a specialized electronic device used to regulate the speed, direction, and braking of an electric motor. Acting as the interface between a power source, a control signal, and the motor itself, the ESC converts low-power control inputs—typically a pulse-width modulation (PWM) signal from a microcontroller or receiver—into high-power switching signals that drive the motor. ESCs are commonly used with both brushed and brushless DC motors in applications such as robotics, drones, electric vehicles, and radio-controlled systems. By rapidly switching power to the motor and precisely controlling the timing of that switching, the ESC allows a control system to smoothly accelerate, decelerate, and maintain motor speed while efficiently managing electrical power.

4.4.2 ESC for Brushed DC Motor

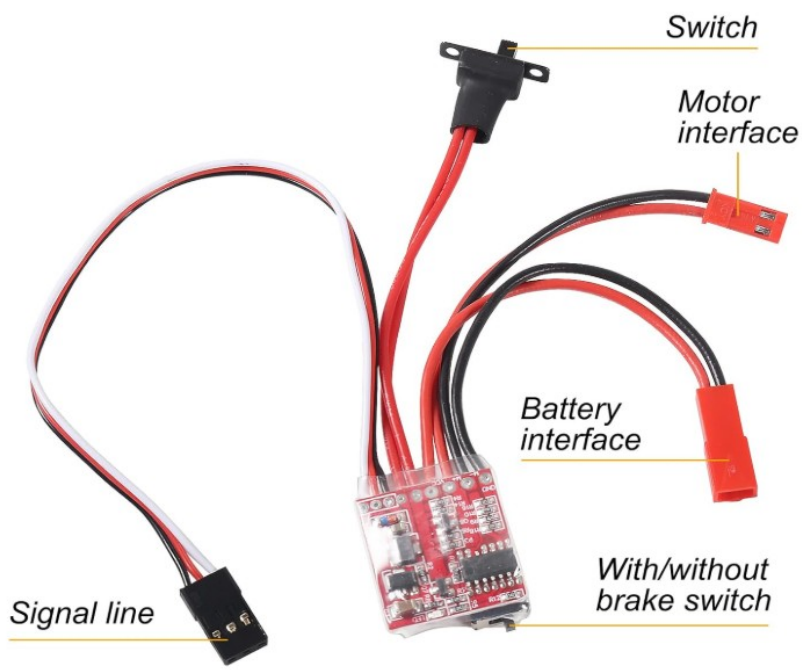


Figure 4.25: ESC Brushed DC Motor

(See Figure 4.25) Connections Explained:

- *Switch* - On/Off

- *Motor Interface* - To Motor
- *Battery Interface* - To 2S Battery
- *Signal Line White* - Input Control PWM (Servo Signal)
- *Signal Line Black* - Ground/Reference
- *Signal Line Red* - This is an 5V regulated output that can be used to power an Arduino or ESP32. **<u>DO NOT</u> connect Voltage!!** With nothing connected to the red pin, measure using a DMM to verify regulator voltage before connecting to a micro-controller.

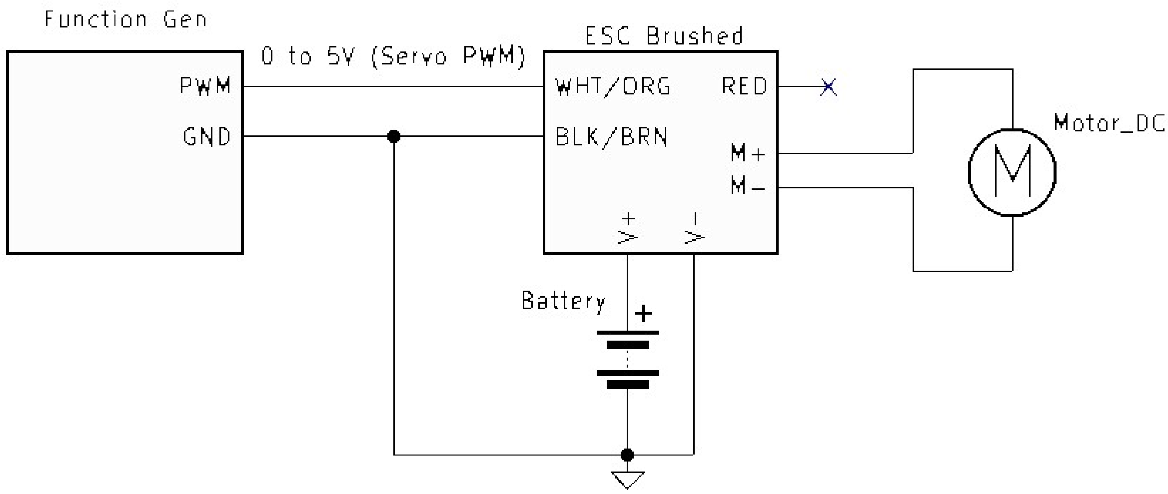


Figure 4.26: Function Generator ESC Brushed DC Motor Test Circuit Schematic

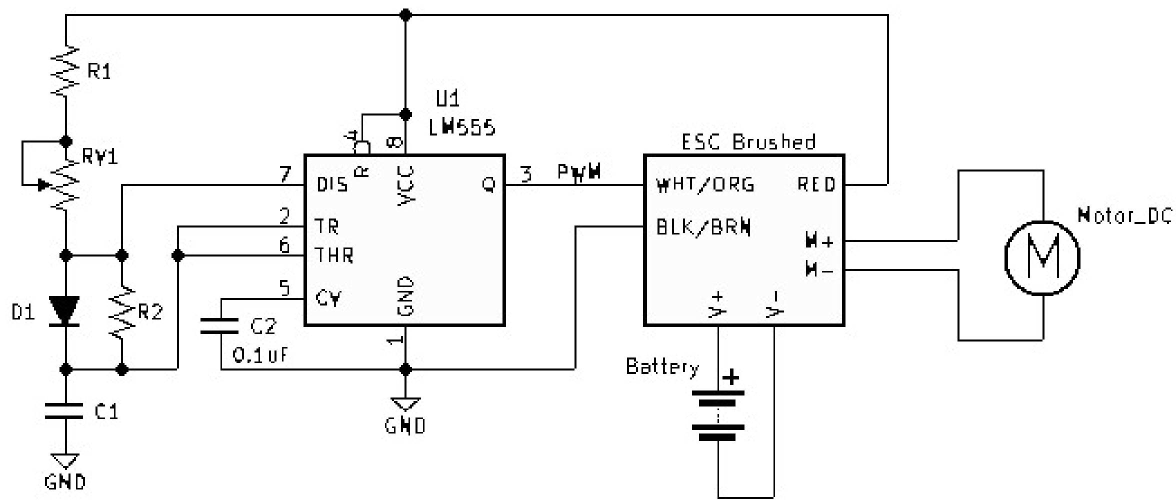


Figure 4.27: 555 Timer ESC Brushed DC Motor Test Circuit Schematic

4.4.3 ESC for Brushless DC Motor

Figure 4.28: ESC Brushless DC Motor

(See Figure 4.28) Connections Explained:

- *Red/Black* - Battery Interface/Input Voltage
- *Three Black Wires* - To Brushless Motor

- *Signal Line White* - Input Control PWM (Servo Signal)
- *Signal Line Black* - Ground/Reference
- *Signal Line Red* - This is an 5V regulated output that can be used to power an Arduino or ESP32. **<u>DO NOT</u> connect Voltage!!** With nothing connected to the red pin, measure using a DMM to verify regulator voltage before connecting to a micro-controller.

Figure 4.29: ESC Bi-directional Brushless DC Motor

(See Figure 4.28) Connections Explained:

- *Red/Black* - Battery Interface/Input Voltage
- *Three Black Wires* - To Brushless Motor
- *Signal Line Orange* - Input Control PWM (Servo Signal)
- *Signal Line Brown* - Ground/Reference
- *Signal Line Red* - This is an 5V regulated output that can be used to power an Arduino or ESP32. **<u>DO NOT</u> connect Voltage!!** With nothing connected to the red pin, measure using a DMM to verify regulator voltage before connecting to a microcontroller.

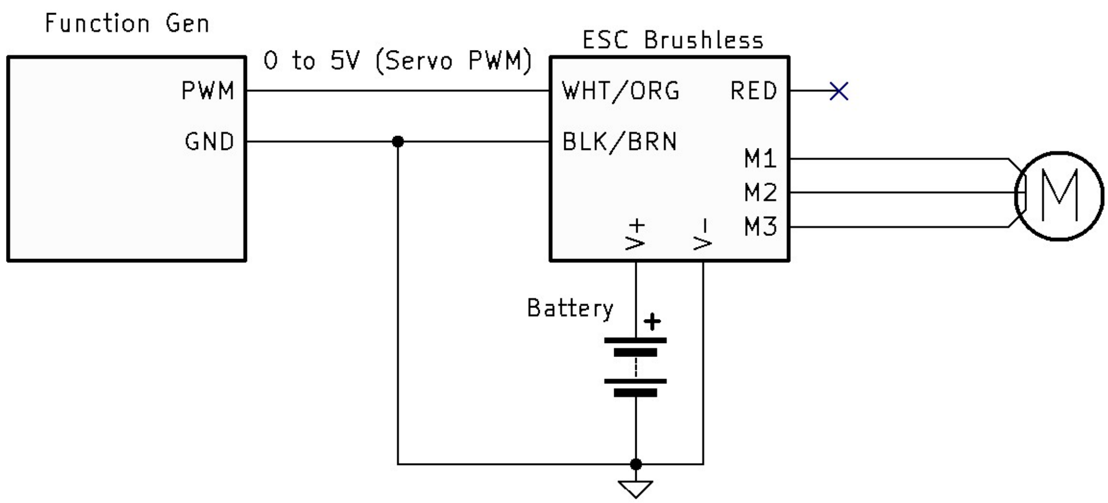


Figure 4.30: Function Generator PWM ESC Test Circuit Schematic

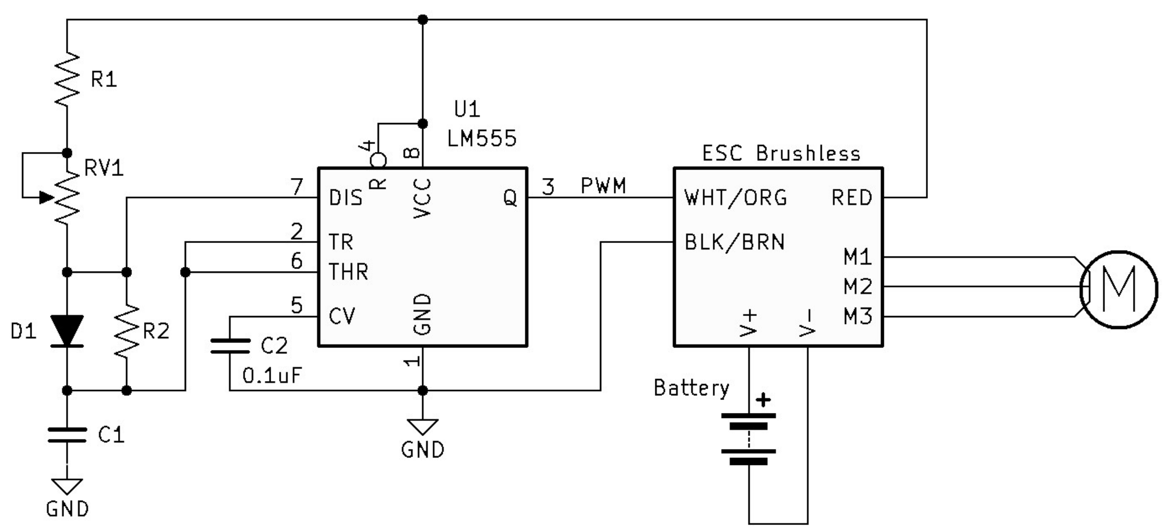


Figure 4.31: 555 Timer PWM ESC Test Circuit Schematic

References